

Continuous Integration (CI) – CURRENCY CONVERSION

Objective:

You should implement a complete DevOps CI pipeline for the **Currency Conversion Java application** provided in the below repository:

[Source Code Repository – final_project](#)

You should **not create a new Java application**.

You should clone the above application, create your own GitHub repository, make the required changes, and implement the complete DevOps pipeline.

1. Clone the Source Code

You should initially clone the trainer-provided repository to your Linux/EC2 machine.

```
git clone https://github.com/devvikasmanda/final_project.git
```

Go inside the project:

```
cd final_project
```

Check the files:

```
ls -la
```

Check Git status:

```
git status
```

2. Create Your Own GitHub Repository

You should create your **own GitHub repository** in **your own github account**.

Recommended repository name:

currency-conversion-devops

Your final project should be maintained in your own GitHub repository.

You should remove the trainer repository as the `origin` and configure your own repository.

```
git remote -v
```

Remove existing origin:

```
git remote remove origin
```

Add your own repository:

```
git remote add origin <YOUR_GITHUB_REPOSITORY_URL>
```

Verify:

```
git remote -v
```

3. Git Initial Setup

You should configure your Git username and email.

```
git config --global user.name "<YOUR_NAME>"
```

```
git config --global user.email "<YOUR_EMAIL>"
```

Verify:

```
git config --list
```

4. Git Branching Strategy

You should maintain the following branches:

main
develop
feature/*
release/*

Recommended flow:

feature
↓
develop
↓
release
↓
main

You should create the required branches.

5. Git Commit Practice

You should not push the entire project using a single commit.

You should make a minimum of 5 meaningful **commits**.

For example:

Initial currency conversion application
Added application changes
Added unit test changes
Added Dockerfile
Added Jenkinsfile
Added shell scripts
Updated README

You should demonstrate:

```
git status
git add
git commit
git log
git log --oneline
git diff
```

6. Git Advanced Operations

You should demonstrate the following as part of the project:

Git Merge

You should create a feature branch and merge it into develop.

Git Rebase

You should rebase your feature branch with the latest develop.

Git Squash

You should create multiple feature commits and squash them into one meaningful commit.

```
git rebase -i HEAD~5
```

Git Cherry-pick

You should create a separate commit and cherry-pick that specific commit into another branch.

```
git cherry-pick <commit-id>
```

Git Stash

You should create uncommitted changes and temporarily store them.

```
git stash
git stash list
git stash apply
git stash pop
```

Git Revert

You should demonstrate reverting a previously committed change.

```
git revert <commit-id>
```

7. Push Your Code to GitHub

You should push your project to your own GitHub repository.

```
git push -u origin main
```

You should also push your other branches.

```
git push -u origin develop
git push -u origin feature/<feature-name>
```

8. GitHub Pull Request

You should create a Pull Request from:

feature → develop

You should demonstrate:

- Pull Request creation
- Code review
- PR approval
- Merge
- Branch management

You should also demonstrate at least **one merge conflict and its resolution**.

9. Linux Requirements

You should use the Linux/EC2 machine for the complete project.

You should demonstrate the Linux commands covered in the training, including:

pwd
ls
ls -l
ls -a
ls -lrt
cd
cat
less
more
head
tail
tail -f
touch
cp
mv
rm
mkdir
grep
find
chmod
chown
chgrp
df
du
tar
gzip

You should use these commands wherever applicable to the project instead of simply running them for demonstration.

10. Shell Scripting

You should create a `scripts` directory:

`scripts/`

You should create **at least 3 shell scripts**.

Recommended:

`scripts/`

- |— `server_check.sh`
- |— `log_check.sh`
- |— `archive_logs.sh`

11. `server_check.sh`

You should create a script that displays:

- Current User
- Hostname
- IP Address
- Current Date
- Disk Usage
- Memory Usage
- Running Processes

The script should use:

- variables
- commands
- if/else
- exit codes

12. `log_check.sh`

You should create an application log:

logs/application.log

The script should:

- Check whether the log exists.
- Search for ERROR.
- Search for WARN.
- Count errors.
- Display the latest logs.

You should use:

grep
wc
tail
if

13. archive_logs.sh

You should create a real-time log archival script.

The flow should be:

Application Logs



Find old logs



Compress



Move to archive



Transfer to archive server

You should use the Linux commands covered in the course:

find
gzip
tar

mv
sftp

14. Cron Job

You should schedule your log archival script using Cron.

You should demonstrate:

crontab -e
crontab -l

Your cron job should automatically execute `archive_logs.sh`.

15. Maven

You should use the existing Currency Conversion application and make sure it builds successfully with Maven.

You should verify:

mvn --version

Then execute:

mvn clean
mvn validate
mvn compile
mvn test
mvn package

You should generate a JAR under:

target/

You should verify the generated artifact.

16. Maven Dependency Management

You should understand and demonstrate the `pom.xml`.

Your project should contain:

`pom.xml`

You should identify:

`groupId`
`artifactId`
`version`
`dependencies`
`plugins`

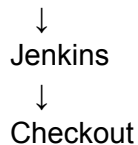
17. Jenkins

You should configure Jenkins for your own GitHub repository.

The Jenkins pipeline should automatically obtain your code from GitHub.

The flow should be:

Your GitHub Repository



18. GitHub Webhook

You should configure a GitHub webhook.

The expected flow is:

You make code change



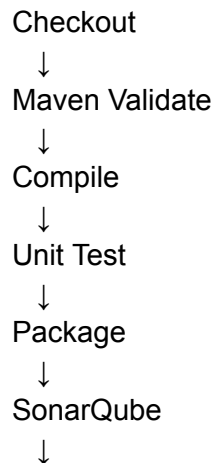
You should demonstrate that the Jenkins build starts automatically after a GitHub push.

19. Jenkins Pipeline

You should create a:

Jenkinsfile

The Jenkins pipeline should contain the following stages:



Quality Gate
↓
JFrog
↓
Docker Build
↓
Trivy Scan
↓
Docker Hub Push

20. SonarQube

You should integrate SonarQube with Jenkins.

Your source code should be analyzed by SonarQube.

You should verify:

Bugs
Vulnerabilities
Code Smells
Coverage
Quality Gate

21. Quality Gate

You should configure a Quality Gate.

The pipeline should behave like this:

SonarQube
↓
Quality Gate
↓
PASS
↓
Continue

If Quality Gate fails:

SonarQube
↓
Quality Gate
↓
FAIL
↓
STOP PIPELINE

JFrog and Docker stages should not execute when the Quality Gate fails.

22. JFrog Artifactory

After successful SonarQube Quality Gate, you should upload the Maven artifact to JFrog.

Expected flow:

Maven
↓
target/*.jar
↓
Jenkins
↓
JFrog Artifactory

You should verify the JAR exists in JFrog.

23. Dockerfile

You should create:

Dockerfile

The Dockerfile should package the Currency Conversion application.

You should use the Dockerfile instructions covered in the training, such as:

```
FROM  
WORKDIR  
COPY  
RUN  
EXPOSE  
CMD / ENTRYPOINT
```

24. Docker Image

Jenkins should build the Docker image.

Example:

```
docker build -t currency-conversion:1.0 .
```

You should verify:

```
docker images
```

25. Docker Container

You should run the application as a container.

You should demonstrate:

```
docker run  
docker ps  
docker ps -a  
docker logs  
docker exec  
docker stop  
docker rm
```

You should also demonstrate port mapping.

Example:

```
docker run -d -p 8080:8080 currency-conversion:1.0
```

26. Trivy

Before pushing the Docker image to Docker Hub, you should scan the image using Trivy.

```
trivy image currency-conversion:1.0
```

You should identify:

CRITICAL
HIGH
MEDIUM
LOW

You should fix the relevant vulnerabilities where possible and perform the scan again.

27. Docker Hub

Create one docker hub account and create a repository.

Only after the Docker image passes the required Trivy validation, you should push the image to Docker Hub.

You should:

Docker Login

↓

Docker Tag

↓

Docker Push

Example:

docker login

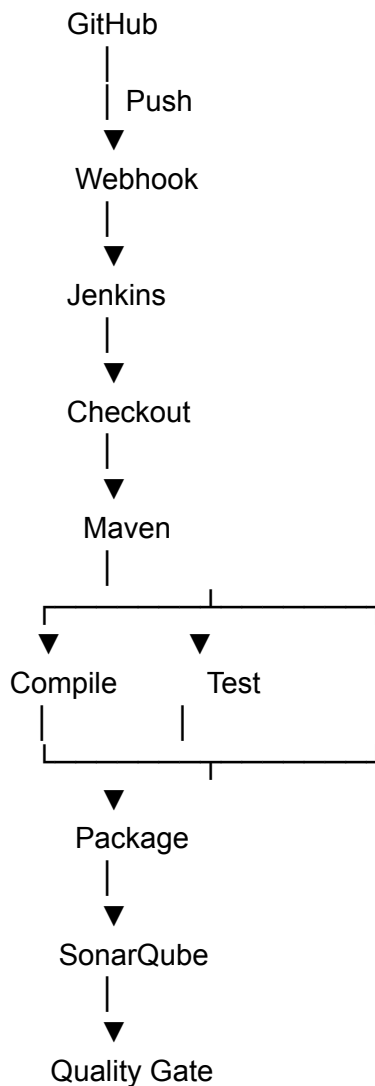
docker tag currency-conversion:1.0 <dockerhub-username>/currency-conversion:1.0

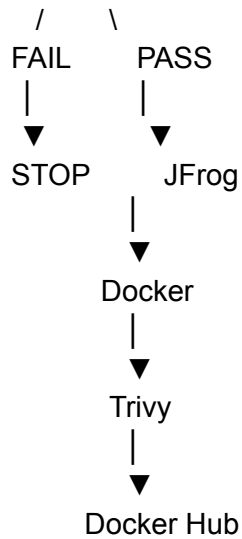
docker push <dockerhub-username>/currency-conversion:1.0

You should verify the image in your Docker Hub repository.

28. Final Jenkins Pipeline

Your final pipeline should work like this:





29. Final GitHub Repository Structure

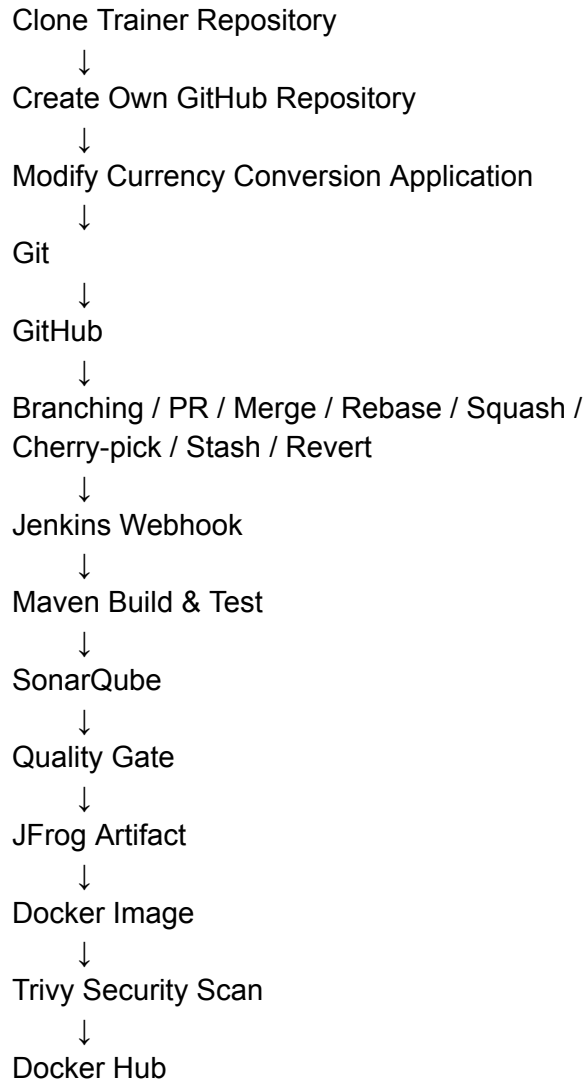
Your final repository should look approximately like:

currency-conversion-devops/

```
|
|— src/
|   |— main/
|   |— test/
|
|— scripts/
|   |— server_check.sh
|   |— log_check.sh
|   |— archive_logs.sh
|
|— logs/
|
|— Dockerfile
|— Jenkinsfile
|— pom.xml
|— README.md
|— .gitignore
```

30. Final Requirement

The final project should demonstrate the complete journey:



Important

You should **start from source code**:

https://github.com/devvikasmanda/final_project